

《通信系统与网络》

信息科学与工程学院

2026.03.30



第八讲:

边缘计算



- 1 云端端协同与边缘计算
- 2 智能驾驶中的边缘服务器
- 3 视频读取与显示
- 4 图片的尺寸调整与格式转换
- 5 Jpeg编解码
- 6 实验步骤

1

云端端协同与边缘计算

2

智能驾驶中的边缘服务器

3

视频读取与显示

4

图片的尺寸调整与格式转换

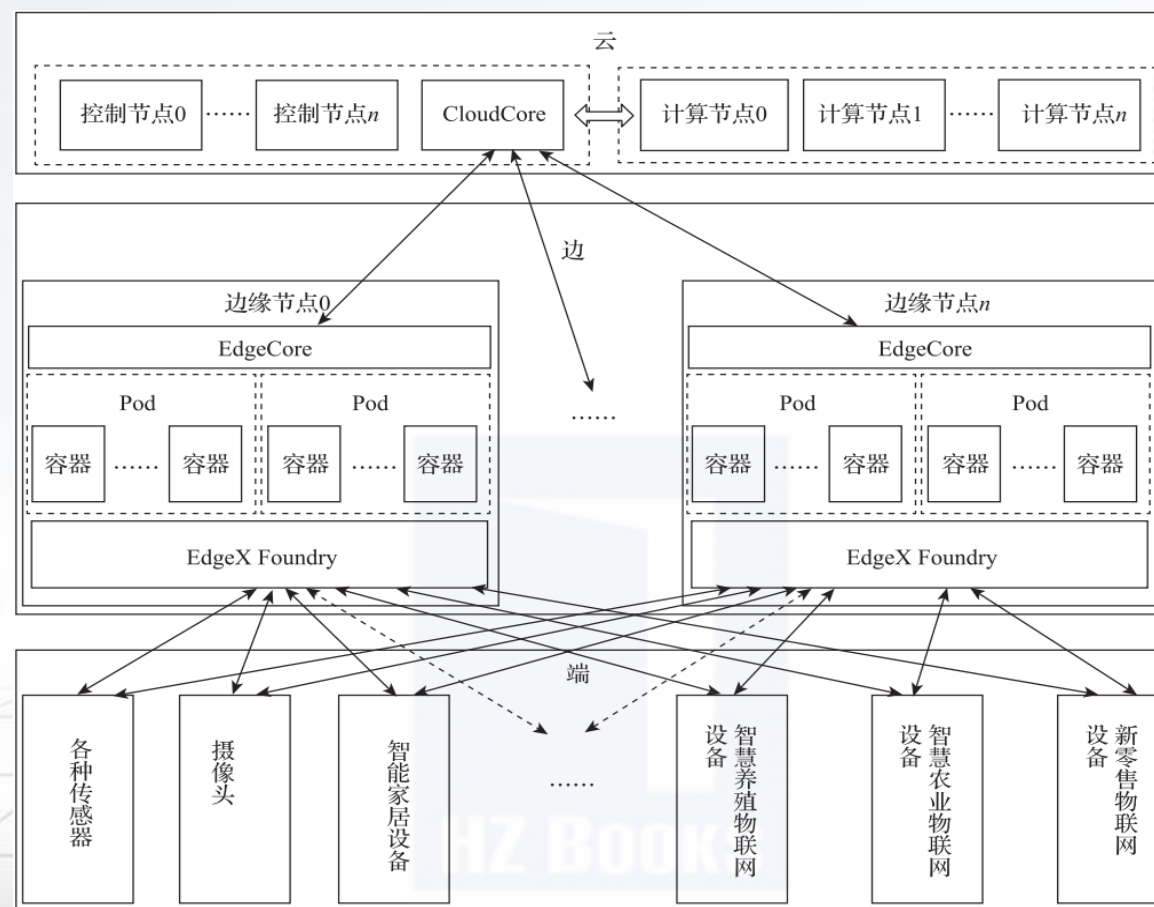
5

Jpeg编解码

6

实验步骤

- 大型网络普遍采用云-边-端架构。
- 云边端架构是一种分布式计算架构，将计算、存储、处理能力拆分到云端、边缘节点、终端设备三个层级，实现分工协作与协同联动，以解决传统云中心化架构在时延、带宽及隐私方面的局限。
- 核心在于“数据在哪里，处理就在哪里”，既保留云端的全局调度能力，又发挥边缘和终端的本地处理优势。



- **云端 (Cloud)**：算力中心，承载重计算任务、数据持久化存储、全局资源调度等功能。其特征为高并发、高可靠、弹性伸缩。
- **边缘 (Edge)**：就近服务，进行实时交互响应、本地数据缓存、协议优化加速等功能。其特征为低延迟、高带宽、就近接入。
- **终端 (Device)**：轻量接入，完成显示与输入、轻量本地计算、安全边界控制等功能。其特征为极简硬件、即插即用。

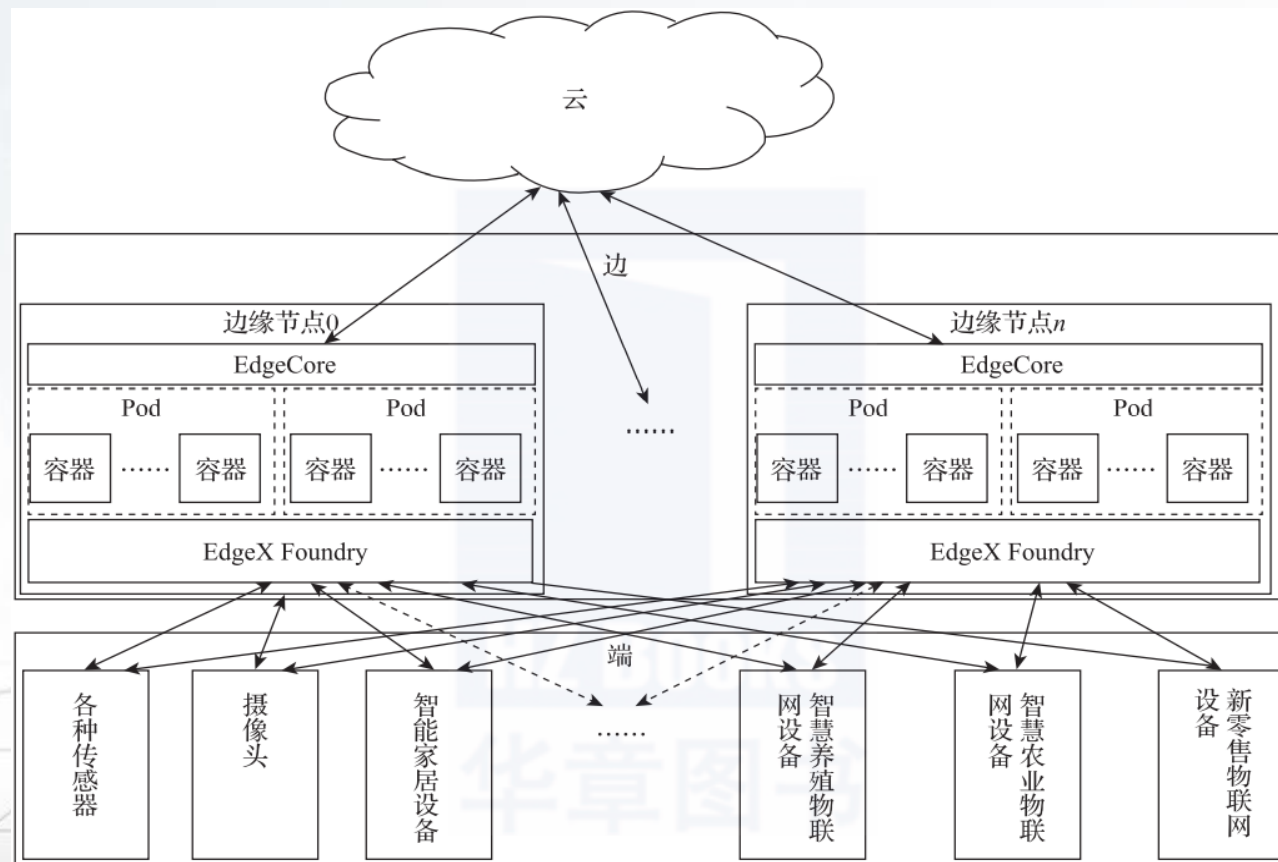
为什么需要云边端协同？

- (1) **单一云架构的局限**：地理距离导致网络延迟，海量并发有带宽瓶颈，单点故障有网络风险。
- (2) **单一边缘架构的局限**：算力规模受限，存储容量有限，运维分散复杂。
- (3) **协同架构的优势**：边缘处理交互任务，降低延迟；云端承载重负载，保障性能；终端轻量化，降低成本。

应用场景：

- **工业互联网：**实时采集设备运行数据、本地判断故障，云端统筹调度优化生产。
 - **智慧城市：**智能交通、安防监控中，边缘节点本地处理违章识别与车流监测，云端优化交通调度。
 - **具身智能：**在机器人领域，云端部署大模型（VLM）负责常识推理，边缘侧处理视觉流与轨迹生成，端侧负责毫秒级运动控制。
 - **其他领域：**包括低空智联网、数字化电力台区治理、储能系统管理及无人装备体系等。
-
- 随着5G网络的普及和边缘计算节点的规模化部署，云边端协同架构将从概念走向大规模商用。
 - 企业IT架构设计，需要重新审视“云优先”策略，转向“场景驱动、分层设计”的新范式。
 - 云边端协同不是技术堆叠，而是面向不同业务场景的精细化架构设计。
 - 理解每一层的技术特性和适用边界，是构建高效、经济、可靠IT系统的基础。

- **边缘计算（Edge Computing）**，是一种分布式计算框架，指在靠近物或数据源头的网络边缘侧，融合网络、计算、存储、应用核心能力的开放平台，就近提供边缘智能服务。
- 端设备的数据处理请求在边缘侧完成，以满足行业在实时业务、应用智能等方面的需求，重点考虑的端边协同。
- 可应用于智能交通、能源分析、物流规划等场景，处理过程在本地完成，能提升效率并减轻云端负荷。



1

云边端协同与边缘计算

2

智能驾驶中的边缘服务器

3

视频读取与显示

4

图片的尺寸调整与格式转换

5

Jpeg编解码

6

实验步骤

智能驾驶系统的边缘服务器是支撑车路协同和自动驾驶实时决策的关键基础设施，部署在网络边缘（如路侧、园区、路口等），靠近数据源与终端设备。

核心功能：

（1）低时延数据处理与响应

- 接收来自车载传感器（摄像头、雷达、激光雷达等）和路侧设备的原始数据，在本地完成目标检测、轨迹预测、场景理解等计算，将端到端时延压缩至10 - 100ms，满足自动驾驶对实时性的严苛要求³⁵。

（2）车路协同（V2X）支持

- 通过路侧单元（RSU）实现车-路-云信息交互，向自动驾驶车辆广播交通信号状态、障碍物预警、道路事件等信息，弥补单车感知盲区，提升行车安全¹¹³。

（3）本地闭环数据处理与隐私保护

- 敏感数据（如车辆位置、行人轨迹）在本地完成采集、分析与存储，无需上传云端，避免数据泄露风险，符合工业与交通领域的数据合规要求¹⁴。

（4）高并发接入与算力聚合

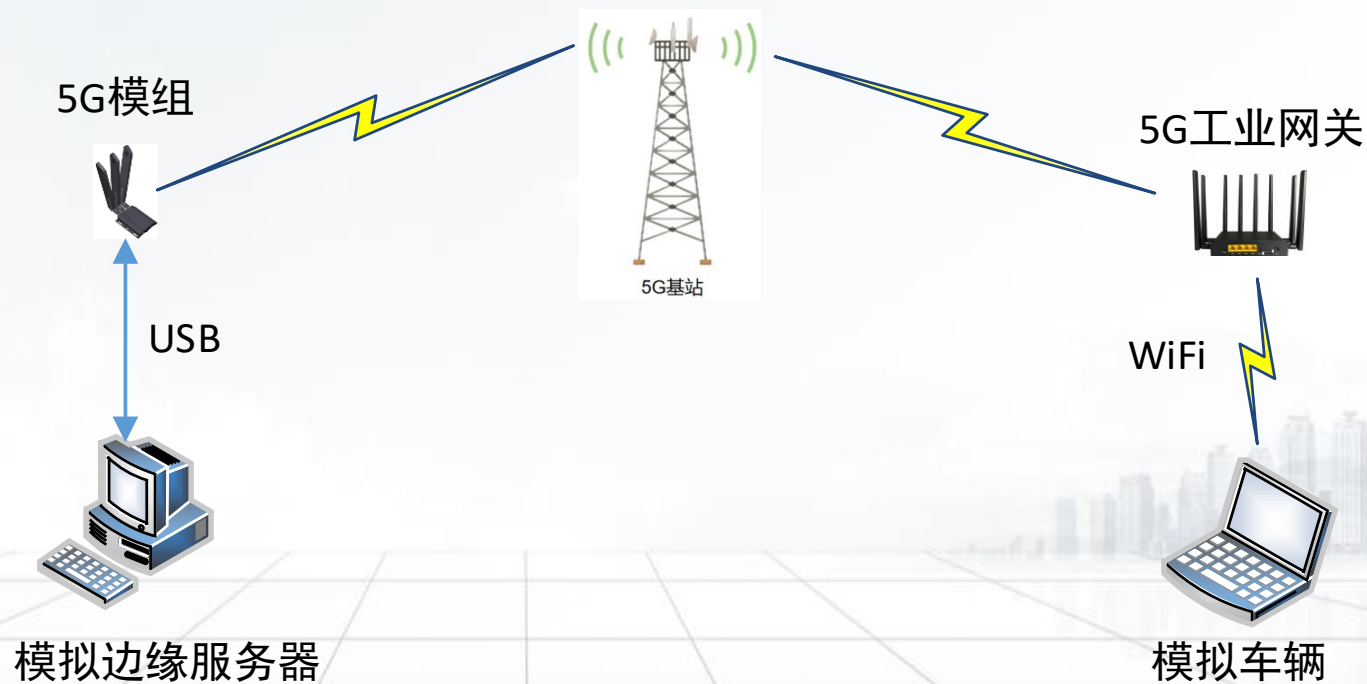
- 支持多路传感器接入（如双向8车道路口），集成AI加速卡（如NVIDIA Orin、Jetson AGX），提供10 - 275+ TOPS算力，实现高精度（位置误差 $\leq 1.0\text{m}$ ）的全要素交通感知¹¹⁵。

（5）边缘-云协同与模型迭代

- 仅将关键摘要或异常事件上传云端，减轻网络负载；同时接收云端下发的优化模型，通过联邦学习或增量学习持续提升本地AI性能。

- 本实验中，我们用已经实现的基本5G数传系统，增加相应的软件，模拟智能驾驶场景下车辆将视频信息送往边缘服务器进行处理的过程
- 实验室计算机模拟边缘服务器，笔记本电脑模拟车辆。
- 车辆采集视频并送往边缘服务器进行计算，再将计算结果发送回车辆。

采用这种不太合理的方式主要是为了使用笔记本电脑自带的摄像头。



1

云边端协同与边缘计算

2

智能驾驶中的边缘服务器

3

视频读取与显示

4

图片的尺寸调整与格式转换

5

Jpeg编解码

6

实验步骤

(1) VideoCapture类

VideoCapture是一个类，提供了从视频源捕获帧的接口。

视频源类型：

- 视频文件，如avi, mp4, mkv等格式；
- 图像序列，如img01.jpg, img02.jpg.....
- 摄像头，如电脑自带摄像头，USB摄像头，网络摄像头
- 视频流，如RTSP, HTTP

以下语句创建了一个VideoCapture实例对象，并从本机默认摄像头（通常是系统自带的）读取视频帧：

```
cap = cv2.VideoCapture(0)
```

(2) read方法

Read()是VideoCapture类的成员函数，是OpenCV中最重要的方法，用于捕获、解码并返回下一个视频帧，格式为：

```
ret, frame = cap.read()
```

其中，ret是一个布尔型成功标志。帧读取成功，ret为True，否则为False；frame是一个numpy格式的数组，存储了视频帧图像。该数组是三维的（高度、宽度、通道数BGR）。

(3) imshow方法

- 在指定窗口中显示图像，格式通常为Mat类型或Numpy数组。须配合waitKey()使用，否则无法正常显示。

`cv2.imshow(winname, mat)`

- 其中，winname是字符串，用于标志一个窗口，作为窗口的标题出现。Mat是要显示的图像。

(4) waitKey方法

- OpenCV 库中关键函数，用于控制图像或视频窗口显示时间，并响应键盘输入事件。格式为：

`waitKey (delay)`

- 通常与 cv2.imshow() 配合，防止图像窗口一闪而过。
- 作用：等待指定毫秒数（delay）内是否有按键按下。若无按键，则返回-1且继续执行后续代码；若有按键按下，返回该键的 ASCII 码值（如 'q' 对应 113）。
- 在等待期间会暂停程序执行，同时激活所有由当前进程创建的 OpenCV 窗口。
- 如果delay=0，则会无限等待，直至按下一个按键。
- 在播放视频时，通常delay=40，因为视频的帧速率一般为25，所以每帧持续40ms正好可以实现流畅播放。

(5) release方法

- VideoCapture的成员函数，用于关闭摄像头或视频文件链接，释放相关资源。
- 如果不调用此方法，将会导致摄像头被占用，下次无法打开。

(6) putText方法

在图像上绘制文本。

```
cv2.putText(img, text, org, fontFace, fontScale, color, thickness=None, lineType=None, bottomLeftOrigin=None)
```

参数说明

- **img**: 目标图像 (需为 NumPy 数组) ; **text**: 要绘制的字符串 (不支持中文, 否则显示为乱码或问号) ;
- **org**: 文本左下角坐标 (x, y), 以像素为单位;
- **fontFace**: 字体类型, 常见选项包括:
 - cv2.FONT_HERSHEY_SIMPLEX; cv2.FONT_HERSHEY_PLAIN; cv2.FONT_HERSHEY_COMPLEX;
 - cv2.FONT_HERSHEY_TRIPLEX; cv2.FONT_HERSHEY_SCRIPT_SIMPLEX; cv2.FONT_HERSHEY_SCRIPT_COMPLEX.
 - 可与 cv2.FONT_ITALIC 组合实现斜体 (如 cv2.FONT_HERSHEY_TRIPLEX + cv2.FONT_ITALIC) 。
- **fontScale**: 字体缩放比例 (数值越大, 字体越大) 。
- **color**: 文本颜色, 格式为 BGR 元组 (如 (255, 0, 0) 表示蓝色) 。
- **thickness**: 字体粗细 (默认为 1) 。
- **lineType**: 线条类型 (推荐使用 cv2.LINE_AA 抗锯齿效果更平滑) 512。
- **bottomLeftOrigin**: 是否以左下角为坐标原点 (通常不使用, 保持默认 None)

注: putText不支持中文。如要在图像上添加中文字符, 可使用PIL (pillow) , 因略复杂此处不介绍, 有兴趣的同学自行学习。

1

云边端协同与边缘计算

2

智能驾驶中的边缘服务器

3

视频读取与显示

4

图片的尺寸调整与格式转换

5

Jpeg编解码

6

实验步骤

(1) resize方法

- 用于调整图像尺寸。支持多种插值方式，适用于缩小、放大等场景。

`dst = cv2.resize(src, dsize[, dst[, fx[, fy[, interpolation]]])`

- 其中，src是输入图像，必须存在；dsize为目标图像尺寸，格式为 (width, height)，若为None，则需要通过fx和fy指定比例；fx/fy：水平和垂直方向的缩放比例，默认值为None；interpolation是插值方法。

➤ 插值方法

方法	适用场景	特点
INTER_NEAREST	快速缩放、实时处理	速度最快， 但易产生像素化
INTER_LINEAR	默认推荐（放大/ 缩小通用）	平衡速度与质量
INTER_AREA	缩小图像（推荐）	避免锯齿，保留细节更好
INTER_CUBIC	放大图像（追求质量）	质量较好，速度较慢
INTER_LANCZOS4	高质量缩放（慢）	质量最优，速度最慢

若不指定，插值方法默认为INTER_LINEAR。

(2) cvtColor方法

- 图像颜色空间转换核心函数。支持多种色彩模型相互转换，例如 BGR ↔ 灰度、BGR ↔ HSV、RGB ↔ YUV 等。

`dst = cv2.cvtColor(src, code[, dst[, dstCn]])`

- 其中，src：输入图像（如 BGR 格式的彩色图像）；code：颜色转换标识符（如 cv2.COLOR_BGR2GRAY）；dst：输出图像（可选，通常由函数自动创建）；dstCn：目标图像通道数（默认为 0，表示自动推断）。注意OpenCV 默认以 BGR 顺序读取和存储彩色图像，而非常见的 RGB 顺序。

- 常见颜色转换标识符：

- cv2.COLOR_BGR2GRAY：BGR → 灰度图
- cv2.COLOR_BGR2HSV：BGR → HSV
- cv2.COLOR_BGR2RGB：BGR → RGB
- cv2.COLOR_GRAY2BGR：灰度 → BGR（伪彩色）

(3) absdiff方法

- 计算两个图像（或矩阵）逐像素绝对差值，广泛应用于运动检测、背景减除、图像差异分析等场景。

`diff = cv2.absdiff(image1, image2)`

- 像素的插值结果为非负，即取绝对值。要求两个图像有相同的尺寸和数据类型。支持多通道图像，各通道独立处理。

- 1 云端协同与边缘计算
- 2 智能驾驶中的边缘服务器
- 3 视频读取与显示
- 4 图片的尺寸调整与格式转换
- 5 **Jpeg编解码**
- 6 实验步骤

➤ 1) imencode方法

- 将图像（通常为 cv::Mat 或 NumPy 数组格式）编码为指定格式（如 JPEG、PNG 等）的内存字节流。该方法在需要通过网络传输、存入数据库或进行内存处理等场景中非常有用。

retval, buf = cv2.imencode(ext, img[, params])

- Ext: 图像格式扩展名，如 “.jpg”、“.png”、“.webp”；img: 输入图像（cv::Mat 或 NumPy 数组）；params: 可选项，为编码参数，例如压缩质量、压缩级别等，以键值对形式传入；
- Buf: 编码后的图像数据（NumPy 数组）；retval: 布尔值，指示图像编码操作是否成功。

➤ 2) imdecode方法

- imdecode 方法用于从内存中的字节缓冲区（如 NumPy 数组）解码图像数据。

img = cv2.imdecode(buf, flags)

- Buf: 包含图像二进制数据的 NumPy 数组，dtype 必须为 np.uint8；flags: 解码方式，常见值如下：
 - cv2.IMREAD_COLOR（默认）：加载为 BGR 彩色图像，忽略 Alpha 通道。
 - cv2.IMREAD_GRAYSCALE: 加载为灰度图。
 - cv2.IMREAD_UNCHANGED: 按原样加载（包括 Alpha 通道）。

- **3) tobytes方法**
- 不是OpenCV 原生方法，而是 NumPy 数组提供的方法。当使用 OpenCV 读取图像后，得到的是一个 NumPy 数组，可以调用 `.tobytes()` 将其转换为字节数据，以便于网上传输。

```
package = jpeg.tobytes()
```

- **4) frombuffer方法**
- 也是NumPy的方法，将字节流转换为numpy数组，可配合 OpenCV 的 `imdecode` 实现从字节流（bytes）读取图像的功能。

```
frame = np.frombuffer(package, dtype=np.uint8)
```

1

云边端协同与边缘计算

2

智能驾驶中的边缘服务器

3

视频读取与显示

4

图片的尺寸调整与格式转换

5

Jpeg编解码

6

实验步骤

(1) 重新构建上一次实验时已经完成的5G基本数传系统;

(2) 在笔记本电脑上编写车载系统模拟程序，完成如下功能:

- 实时采集摄像头的视频信息，在其上用红色添加组号（如group 7）并显示在窗口1中;
- 将视频帧压缩至400*300像素;
- 压缩后的视频帧变为灰度图，并显示在窗口2中;
- 每两帧采样一帧，编码为jpg图像并转换为字节表示;
- 将字节格式的视频帧发送至边缘服务器;
- 接收服务器回传的差分图像（字节表示），经字节到numpy数组的转换和jpeg解码，恢复成原始形式，并显示在窗口3中。

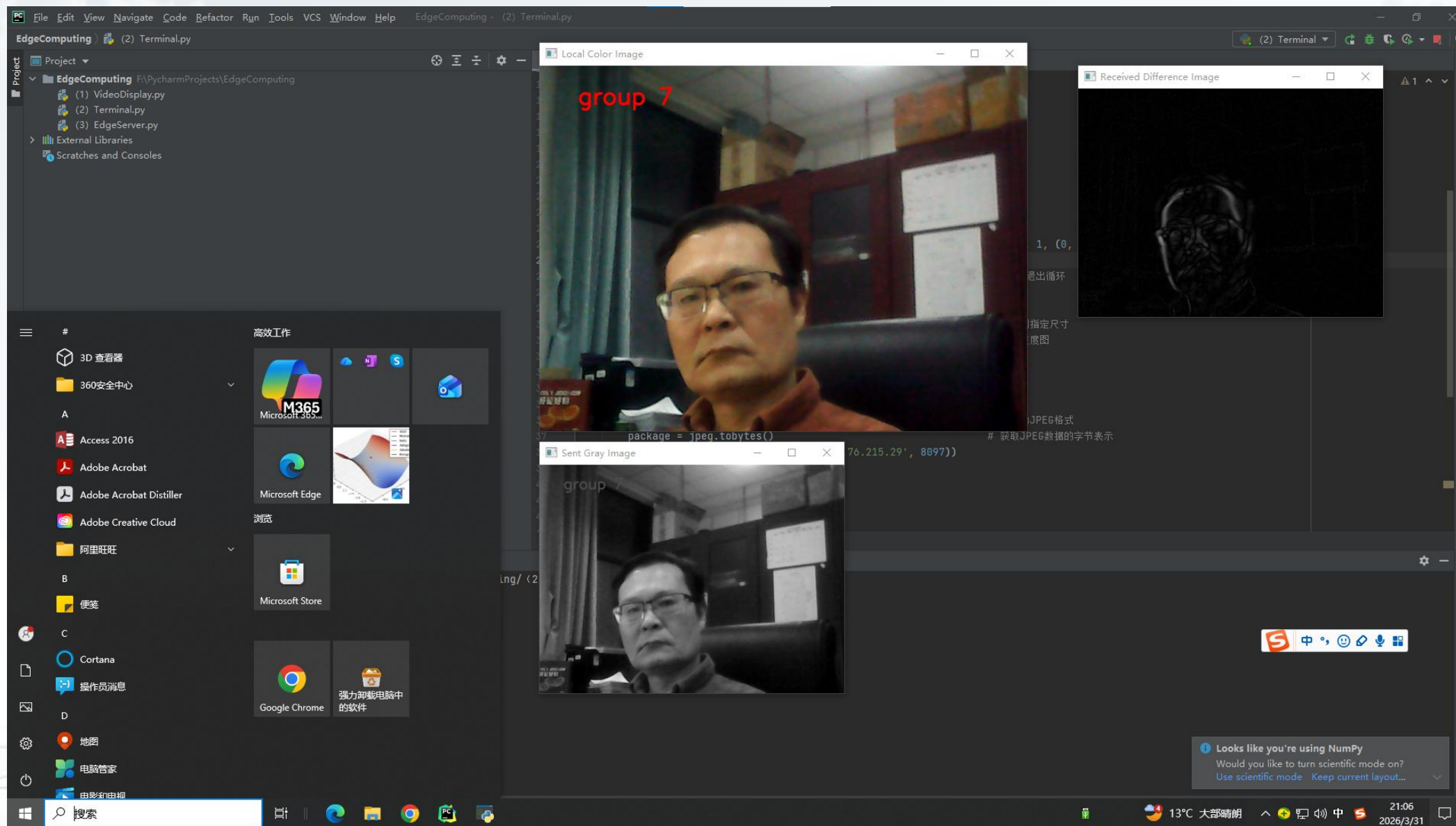
(3) 在实验室计算机上编写边缘服务器程序，完成如下功能:

- 接收车辆发送的视频（字节表示），经字节到numpy数组的转换和jpeg解码，将图像帧恢复成原始形式（400*300的灰度图）;
- 计算差分图像;
- 差分图像经jpeg编码并转化为字节表示后，回传给车辆。

(4) 全部完成后，通知指导教师检查源代码和运行结果，并登记。

- 本实验中将传输的视频帧压缩至400*300，是因为经jpeg编码后的图像可以在一个UDP数据包中放下，因此每帧图像通过一个UDP包传送即可，这可以简化程序的设计，不用考虑需要将一帧图像分为几片，在接收端还需要重新组合成原始图像。
- 实验中可以将UDP数据包的最大长度设置为65507，这个长度可以传输上述jpeg格式的图片。关于UDP数据包的最大长度问题较为复杂，涉及UDP头、IP头、IP层的分片处理、不允许分片标志诸多问题，在此不多做介绍，有兴趣的同学请自行查找资料学习。
- 我们必须意识到，本实验对传输视频的处理仅仅是为了实验方便而做的简化，使大家在有限的时间内可以完成实验，并熟悉OpenCV的基本操作和边缘计算的工作流程。实际的视频传输不会采用这么笨的办法去逐帧传输图片，而是通过推流或拉流的方式直接传输编码后的视频（如H.264格式）。相关内容也请自行学习。

附 截屏效果



END